

Directed Weighted Graphs 1

Dr. Chris Cain

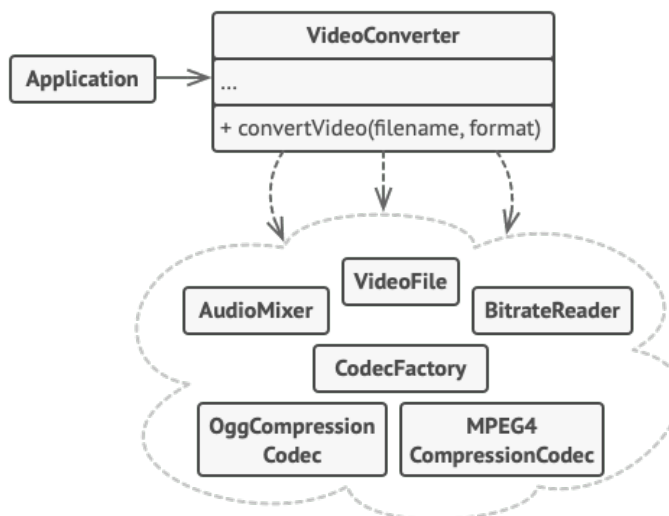
1 Premise

You are a game developer, working on a character-driven RPG game. You want to store and track two pieces of data for every character in your game: relationship and favor. Relationship is what status a given character has with every other character in the world (enemy, stranger, friend, family) while favor is what their opinion is for anyone they know (dislike, neutral, like, love). A character will not have an opinion about someone they haven't met, so you don't need to store a value for every single permutation. However, you do need to store every character's relation with one another, so you can keep track of who knows whom.

This problem is two sides of the same coin: both relationship and favor can be represented as a directed weighted graph, or DWGraph for short. A node in the graph is a character, and each edge is an interaction with another character. The difference is that the favor graph will be sparse, where the relationship graph will be complete (each node is connected to every other node).

2 Design Pattern: Facade

To keep the API the same, no matter what is going on "under the hood", the Facade design pattern needs to be used. This design pattern adds a layer on top of your graph implementation to hide the specific workings from the client code. Facade also allows the data structure to change how it works during runtime.



This example, along with a detailed explanation, can be found at refactoring.guru/design-patterns/facade.

3 Specifications

The relationship and favor problem described in the Premise is what will be used to test your DWGraph implementation. Note that the edge weights are *Doubles*, the object form of a primitive double, and converting from edge weight to some other value is outside of the scope of this assignment.

You need to create 4 classes: the Facade, an interface, and two implementations of the interface. One implementation is an adjacency matrix, the other is an adjacency list. Each of these classes will be their own Java file within the default package (*do not* declare a package at the top of any of your files).

You will need to make a driver for your own testing purposes, but it will not be used to grade your code.

3.1 Structure

The DWGraph will forward its methods to the Digraph interface, and depending on which implementation DWGraph has stored, Digraph will call either AdjMatrix or AdjList. DWGraph is also responsible for determining when to switch its private Digraph from an AdjMatrix to an AdjList (and vice versa).

3.2 DWGraph

Your DWGraph needs to:

- Add a node (return false if node is already in the graph)
- Add an edge (add the nodes if they do not already exist, return false if edge is already in the graph)
- Delete a node and all its edges (outbound and inbound)
- Delete an edge (return null if edge does not exist)
- List all nodes in graph
- List all edges from a node (only outbound edges)
- Get the weight on a specific edge (return null if edge does not exist)
- Calculate the unweighted density of the entire tree
- Calculate the unweighted density of a specific node
- Give the number of nodes in the graph
- Produce a human-readable string representation of the graph
- Produce a JSON serialization of the graph
- Populate the entire graph from a JSON file (skip anything that already exists)
- Automatically swap between using an adjacency matrix and an adjacency list

3.2.1 API Specifications

```
public class DWGraph {  
    private Digraph graph  
    private int size  
    private double mtxThreshold  
    private double lstThreshold  
    public DWGraph()  
    public DWGraph(String filepath)  
    public boolean add(String key)
```

```

    public boolean add(String src, String dest, Double weight)
    public String delete(String key)
    public Double delete(String src, String dest)    //returns the object form of a double
    public ArrayList<String> nodes()
    public ArrayList<String> edges(String key)
    public Double weight(String src, String dest)    //returns the object form of a double
    public double density()
    public double density(String key)
    public int size()
    public String toString()
    public String toJSON()

    public static DWGraph load(String filepath)

    private void convert()
}

```

3.3 Interface Graph

```

interface Digraph {
    boolean add(String key);
    boolean add(String src, String dest, Double weight);
    String delete(String key);
    Double delete(String src, String dest);
    ArrayList<String> nodes();
    ArrayList<String> edges(String key);
    Double weight(String src, String dest);
    double density();
    double density(String key);
    int size();
    String toString();
    String toJSON();
}

```

3.4 Adjacency Matrix

The adjacency matrix needs to store the weights in a primitive 2D array (`Double[][]`), and it also needs to store the key:index mappings (`HashMap<String, int>`).

```

class AdjMatrix implements Digraph {
    private Double[][] weights
    private HashMap<String, int> keyMap
    ...
}

```

3.5 Adjacency List

The adjacency list needs to be implemented with a double-nested `HashMap` for this assignment.

```

class AdjList implements Digraph {
    private HashMap<String, Hashmap<String, Double>> connections
    ...
}

```

3.6 Comments

Each file should have complete Javadoc comments. Javadoc comments are *not* the same as block or inline comments. If you are not familiar with Javadoc comments, review them here: www.oracle.com/technical-resources/articles/java/javadoc-tool.html

3.6.1 Sample Javadoc Comment

```
/**
 * This is an example method to show off Javadoc comments.
 *
 * @param someValue the value to be passed to the method
 * @return          true if method succeeded, otherwise false
 */
public static boolean someMethod(int someValue) {
    ...
}
```

4 Learning Outcomes

By the end of this assignment, students should be able to...

- Identify and justify the Facade design pattern
- Parse the JSON format
- Describe the differences between an adjacency matrix and an adjacency list
- Justify design and implementation choices
- Identify a directed weighted graph

By the end of this assignment, students should have produced...

- A dynamic implementation of a directed weighted graph
- An adjacency matrix implementation
- An adjacency list implementation
- A JSON serialize and de-serialize algorithm

5 Grading

Grading for this assignment is broken down into three main parts, graded out of 240 points.

Points	Section	Description
120	Implementation	Student's code passes all test cases, including edge cases. Code that does not adhere to API specifications (and therefore can't be directly called via the expected method signatures) will receive a zero on this section.
60	Code Conventions	Code is intelligently decomposed (broken into methods to maximize code reuse and readability), Javadoc comments for each method, understandable variable naming, and no magic numbers
60	Reflection	Each file has a block comment (<code>* ... *\</code>) at the top with a response to the file's questions (see below).
48	Extra Credit	Student successfully implements one of the extra credit options (see below). Only one option will be counted.
2	Folder Name	Submitted folder name is both unique in concept and creative. Please keep it school-appropriate.

5.1 Reflection Questions

5.1.1 DWGraph.java

What benefit does using a facade provide? What would happen if this file was omitted from the assignment specifications?

5.1.2 Digraph.java

When did you choose to swap between the matrix and list? Explain and defend your threshold selection process.

5.1.3 AdjMatrix.java

When should you use a matrix to represent a directed weighted graph? Why? Does this reasoning hold true for other types of graphs?

5.1.4 AdjList.java

When should you use a list to represent a directed weighted graph? Why? Does this reasoning hold true for other types of graphs?

5.2 Extra Credit Options

5.2.1 Nodes

Graphs are often drawn out using nodes and edges, but they are seldom implemented that way. Construct a `NodeGraph` (implements `Digraph`) class with private subclasses to represent nodes and edges. Do not use any structures other than nodes and edges to traverse or store the graph. At the top of your file, answer the following questions:

What problems or dilemmas did you encounter while creating this structure? How did you solve them and why did you choose to do it that way? Is there a situation that this implementation of a graph would be better than the matrix or list versions?

5.2.2 Graph Types

A `Digraph` is a broad category of graphs with several subtypes and variants. Using only the Facade design pattern, create at least 2 new facades on top of the `Digraph` for two new types of graphs. Their external API must be the same, except if the graph type does not support it (i.e. if you implement a facade for a directed unweighted graph, `weight(src, dest)` should either return a sensible value or be unimplemented). At the top of each file, answer the following questions:

Why did you select this graph type? How did you manipulate the inputs into the `Digraph` to achieve the implementation? Did you change your thresholds or algorithm for converting between a matrix and a list? If so, why?

6 Submission Guidelines

Submit a single zipped folder with (at least) 4 .java files within it. Please check that the folder unzips correctly (no nested folders, no src folders, etc).

This assignment will take some time. Start early!